

前情提示：

本次实践使用的LangChain版本为1.0，和0.3的版本实现有所不同。

许多库的位置发生了变化。

第一步：文本读取

这一步使用 `langchain_community.document_loaders` 中的 `TextLoader` 实现。

- `TextLoader` 每一次只能读取一个文件，返回的类型是 `List[document]`（只有一个文件但是返回列表的原因是为了统一接口）
 - `document` 类型存在两个属性：
 - `page_content`：存放具体的文本内容；
 - `metadata`：存放元数据，标注来自哪个文件的那一页。

```
1 from langchain_community.document_loaders import TextLoader
2
3 # TextLoader只能读取一个文件，不能批量读取，返回Document列表是为了接口统一，这样不管是什么类型的Loader，只需要修改初始化的对象即可。
4 loader = TextLoader("./藜麦.txt", encoding='utf-8') # 初始化了一个加载器对象，还没有真正读取文件内容。Windows需要设置下格式
5 documents = loader.load() # 真正的读取，读取后得到的是一个 Document 对象列表 (List[Document])
6 # documents[0]是一个 Document 对象
7 # documents[0].page_content 才是具体的文本内容
8 # documents[0].metadata 包含了元数据
9
10 # print(documents)
11 # print("=====")
12 #
13 # print(documents[0].page_content)
14 # print("=====")
15 #
16 # print(documents[0].metadata)
```

第二步：文本分块

这一步是为了避免存入向量数据库中的文本块过长。（文本过长向量化后会有较大的损失，并且要是过长，那么一个块的主题会过多）

```
1 # 第二步：然后进行切块，避免每个块的长度过长
2 from langchain_text_splitters import CharacterTextSplitter,
  RecursiveCharacterTextSplitter
3 text_splitter = RecursiveCharacterTextSplitter(
4     chunk_size=128, # 一个块的大小
5     chunk_overlap=50, # 重叠字段的数量
6     separators=['\n\n', '\n', '。', '!', '?', '//'] # 优化分隔符
7 )
8 texts = text_splitter.split_documents(documents) # 现在不需要设置
  documents[0].page_content和metadatas了，会自动进行
9 # for i, text in enumerate(texts, start=1):
10 #     print(f"现在是第{i}个chunk:\n", text)
11 print(type(texts[0])) # <class 'langchain_core.documents.base.Document'>, 说
  明是document对象
```

使用 `langchain_text_splitters` 中的 `RecursiveCharacterTextSplitter` 进行分块。

先对要使用的文本分块器初始化，然后调用 `.split_documents` 进行切块，这样的好处是数据类型仍然是 `document` 并且包含元数据，这样可以很方便地存入向量数据库。

第三步：使用嵌入模型，将切块后的文本存入向量数据库中

前置：需要先将嵌入模型下载到本地：

```
1 from huggingface_hub import snapshot_download
2
3 local_path = snapshot_download(
4     repo_id="moka-ai/m3e-base",
5     local_dir=r"D:\Code\Agent\models\m3e-base"
6 )
7
8 print("下载完成: ", local_path)
```

可以登录以加速下载。

后续代码：

```
1 # 第三步：使用嵌入模型，将文本转换为数值表示
2 from langchain_huggingface import HuggingFaceEmbeddings
3 from langchain_community.vectorstores import Chroma
4 import os
5
6 # 使用本地模型路径
7 model_path = r"D:\Self_Study\大模型学习\models\m3e-base"
8
9 # 确保模型的路径存在
10 if not os.path.exists(model_path):
11     raise FileNotFoundError(f"模型路径不存在{model_path}，请检查路径！")
12
13 # 配置Embedding模型的参数
14 model_kwargs = {'device':'cpu'} # 使用CPU
15
16 # 创建embedding模型实例
17 embedding = HuggingFaceEmbeddings(
18     model_name=model_path, # 新版本中不是model_path，而是model_name
19     model_kwargs=model_kwargs,
20 )
21
22 db = Chroma.from_documents(texts, embedding=embedding)
23 # print(db.similarity_search("黎一般在几月播种?", k=3))
```

使用 `HuggingFaceEmbeddings` 来对嵌入模型进行初始化，需要传入的有 `model_name`="你的模型路径" 以及 `model_kwargs`={'device':'cpu\GPU'}。

使用 `Chroma` 作为向量数据库，注意，使用 `from_documents` 的时候传入的要为 `List[document]` 对象。

- 若想要保存向量数据库到本地，添加一行参数 `persist_directory="./chroma_db"` # 把 `Chroma` 的数据持久化保存到你指定的本地目录里。即可。

第四步：构建一个能检索本地数据库的Agent

这里使用了LangChain的新函数 `create_agent` 能够很方便地构建一个简单的Agent。

```
1 # 最后完整的流程如下：
2 from langchain.agents import create_agent
3 from langchain.chat_models import init_chat_model
4 from langchain.tools import tool
5
6 api_key = os.environ.get("DEEPSEEK_API_KEY")
7
8 llm = init_chat_model(
9     model = "deepseek-chat",
```

```
10     model_provider='openai',
11     api_key = api_key,
12     base_url="https://api.deepseek.com",
13     temperature = 0
14 )
15
16 retriever = db.as_retriever() # 默认检索方式是向量相似度匹配
17
18 @tool
19 def search_kb(query: str) -> str:
20     """搜索知识库并返回相关内容"""
21     docs = retriever.invoke(query)
22     return "\n\n".join(doc.page_content for doc in docs)
23
24 agent = create_agent(
25     model=llm,
26     tools=[search_kb],
27     system_prompt="你是农业领域的专家，回答问题前优先使用 search_kb 检索相关知识，再基于检索结果回答。"
28 )
29
30 response = agent.invoke(
31     {
32         'messages': [{'role': 'user', 'content': '藜麦怎么防治虫害?'}]
33     }
34 )
35
36 print(response)
37 print("=====")
38 print(response['messages'][-1].content)
```